

Name: Monisha Raghini Chelle

Roll No: 2520030537

Branch: B.Tech CSE

Subject: Linux System Programming Lab

OSSP_Practical3&4

OSSP (Operating Systems And Systems Programming) - 25CS2104E

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        printf("Child is running...\n");

        sleep(30);

        printf("Child process terminating...\n");
    }

    else {
        // Parent process
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child has terminated.\n");
        printf("Parent process terminating...\n");
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^V Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical3.c *
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        printf("Child is running...\n");

        sleep(30);

        printf("Child process terminating...\n");
    }

    else {
        // Parent process
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child has terminated.\n");
        printf("Parent process terminating...\n");
    }

    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

```
Write to File: practical3.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
^C Cancel M-M Mac Format M-P Prepend ^O Discard buffer
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  Program2      Program4.c  Simple-Commands-Week1  program2      program5.c
OSLAB                           Program2.c    Program5    example.txt             program2.c    sample1
OSLAB14                         Program3      Program5.c  foldername              program3      sample1.c
OSSP-Practical                  Program3.c    Program6    hello                   program3.c    sample1.c.s
ave
Program1                        Program4      Program6.c  hello.c                 program4
Program1.c                     Program4.1    Program7    moni                    program4.c
Program1.c.save                Program4.1.c  Program7.c  monis                   program5
monis@DESKTOP-C3GM4T6:~$ cd OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical3.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc practical3.c -o practical3
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical3
```

After approximately 30 seconds:
Before 30seconds

Terminal 1

Run:

`./practical3`

You should see:

Parent PID: XXXX

Child PID: YYYY

Parent is waiting for child...

...

Child is running...

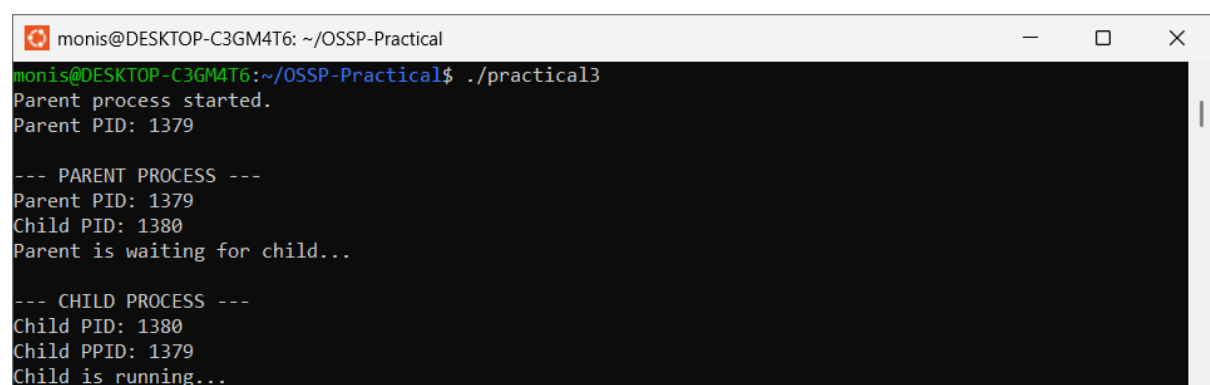
STOP THERE — do not wait for it to finish.

Terminal 2

Immediately run this, replacing XXXX and YYYY with the PIDs printed by Terminal 1:

`ps -o pid,ppid,state,stat,cmd -p XXXX,YYYY`

Terminal -1 :

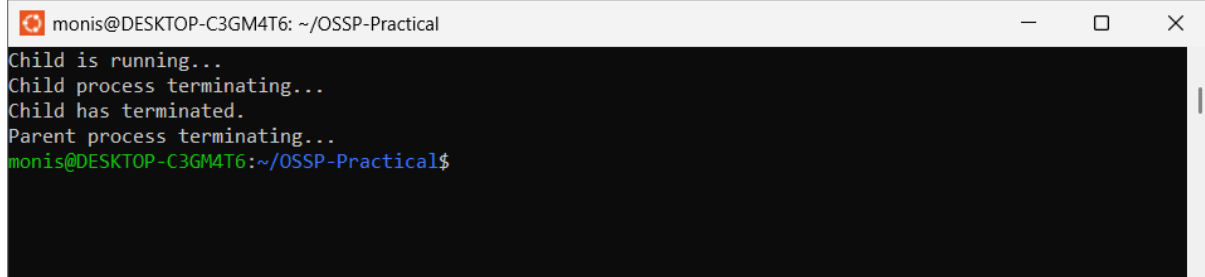


```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical3
Parent process started.
Parent PID: 1379

--- PARENT PROCESS ---
Parent PID: 1379
Child PID: 1380
Parent is waiting for child...

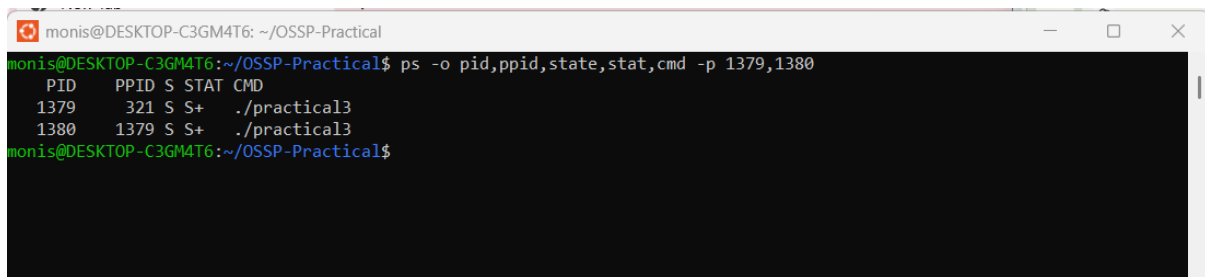
--- CHILD PROCESS ---
Child PID: 1380
Child PPID: 1379
Child is running...
```

After 30seconds (Terminal-1)

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical'. The output shows a child process running, then terminating, followed by the parent process terminating. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$'.

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
Child is running...
Child process terminating...
Child has terminated.
Parent process terminating...
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Go to other terminal (2nd)
[Press -> Window key-> Ubuntu]
cd Documents
cd OSSP-Practical
Terminal-2

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical'. The command 'ps -o pid,ppid,state,stat,cmd -p 1379,1380' is executed, showing two processes: PID 1379 (PPID 321) and PID 1380 (PPID 1379), both in state 'S+'. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$'.

```
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ps -o pid,ppid,state,stat,cmd -p 1379,1380
PID    PPID S  STAT CMD
1379    321 S+   ./practical3
1380    1379 S+   ./practical3
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

2. Compile again

gcc practical3.c -o practical3

3. Terminal 1

Run:

./practical3

When you see:

Child PID: XXXX

Child is running...

do not wait.

4. Terminal 2

Use the new child PID. For example, if it says Child PID: 1500:

cat /proc/1500/status

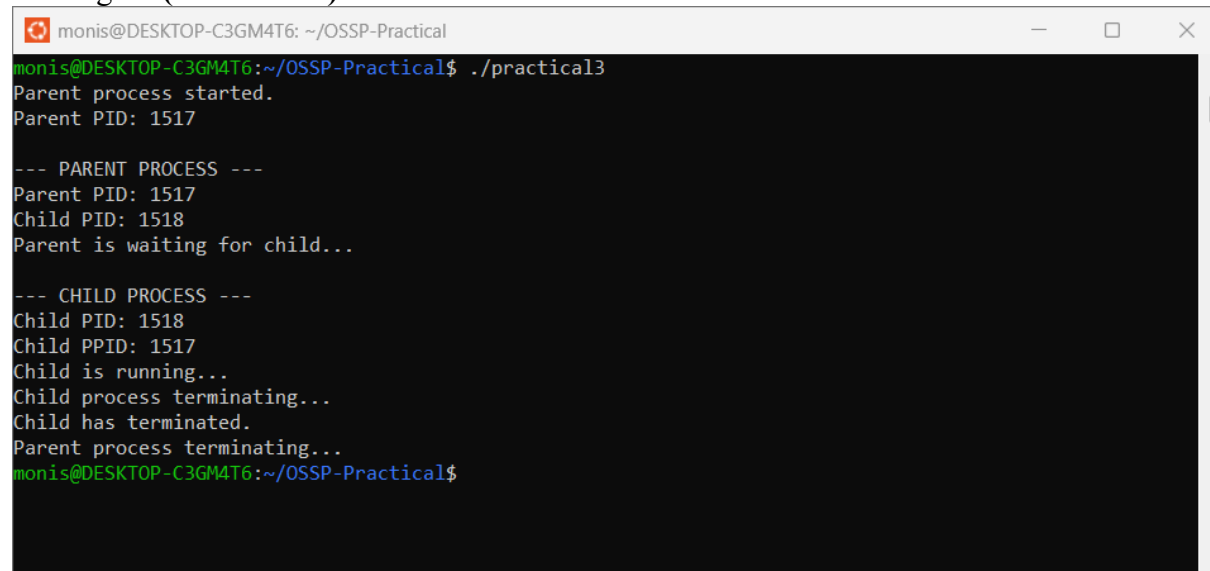
Look for:

State:

Pid:

PPid:

Run Again:(**Terminal-1**)



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical3
Parent process started.
Parent PID: 1517

--- PARENT PROCESS ---
Parent PID: 1517
Child PID: 1518
Parent is waiting for child...

--- CHILD PROCESS ---
Child PID: 1518
Child PPID: 1517
Child is running...
Child process terminating...
Child has terminated.
Parent process terminating...
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Second Terminal run this (below):before run command in Terminal-2

Terminal-2

[illegible]

Then run :top

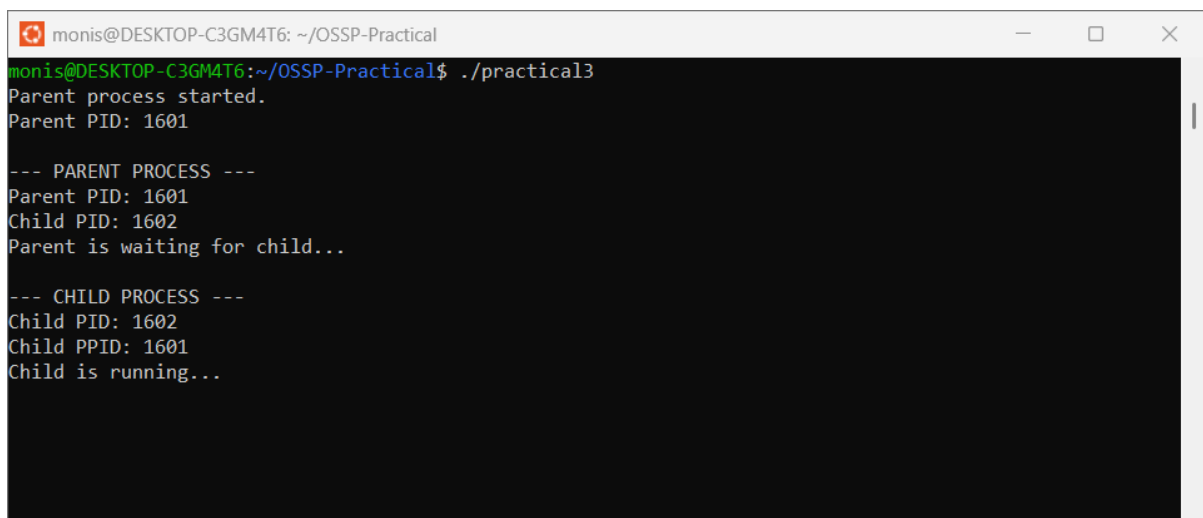
While the program is still running:

top

Find practical3, then press:

q

Again **Terminal-1:**



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical3
Parent process started.
Parent PID: 1601

--- PARENT PROCESS ---
Parent PID: 1601
Child PID: 1602
Parent is waiting for child...

--- CHILD PROCESS ---
Child PID: 1602
Child PPID: 1601
Child is running...
```

Terminal-2

monis@DESKTOP-C3GM4T6: ~/OSSP-Practical

monis@DESKTOP-C3GM4T6:~/OSSP-Practical\$ top

top - 15:33:24 up 1:03, 1 user, load average: 0.00, 0.00, 0.00

Tasks: 30 total, 1 running, 29 sleeping, 0 stopped, 0 zombie

%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st

MiB Mem : 3790.2 total, 3250.4 free, 502.5 used, 172.8 buff/cache

MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 3287.7 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1644	monis	20	0	8180	5760	3712	R	0.3	0.1	0:00.07	top
1	root	20	0	24484	14816	10976	S	0.0	0.4	0:01.84	systemd
2	root	20	0	3120	2048	1920	S	0.0	0.1	0:00.04	init-systemd(Ub
6	root	20	0	3120	1792	1792	S	0.0	0.0	0:00.00	init
49	root	19	-1	50380	16004	14980	S	0.0	0.4	0:00.64	systemd-journal
83	systemd+	20	0	22524	14080	11520	S	0.0	0.4	0:00.16	systemd-resolve
92	root	20	0	35108	11520	8832	S	0.0	0.3	0:02.91	systemd-udev
113	root	20	0	2880	1664	1664	S	0.0	0.0	0:00.09	chronyd-starter
119	root	20	0	4464	2816	2688	S	0.0	0.1	0:00.04	cron
132	message+	20	0	8800	4480	4096	S	0.0	0.1	0:00.14	dbus-daemon
149	root	20	0	44076	28544	13952	S	0.0	0.7	0:00.47	networkd-dispat
153	root	20	0	18420	8960	7936	S	0.0	0.2	0:00.17	systemd-logind
166	syslog	20	0	220532	5248	4224	S	0.0	0.1	0:00.24	rsyslogd
222	_chrony	20	0	24544	11184	6656	S	0.0	0.3	0:00.25	chronyd
229	root	20	0	5528	2688	2560	S	0.0	0.1	0:00.02	agetty
231	root	20	0	123440	31616	17280	S	0.0	0.8	0:00.50	unattended-upgr
233	_chrony	20	0	12076	2232	1664	S	0.0	0.1	0:00.15	chronyd
256	root	20	0	5484	2560	2432	S	0.0	0.1	0:00.01	agetty
315	root	20	0	3136	900	768	S	0.0	0.0	0:00.00	SessionLeader
317	root	20	0	3136	1164	1024	S	0.0	0.0	0:00.25	Relay(321)
321	monis	20	0	6260	5248	3712	S	0.0	0.1	0:00.23	bash
323	root	20	0	8632	5376	4608	S	0.0	0.1	0:00.02	login
373	monis	20	0	22492	13056	10624	S	0.0	0.3	0:00.20	systemd
375	monis	20	0	22892	4044	2048	S	0.0	0.1	0:00.00	(sd-pam)
403	monis	20	0	6128	5120	3712	S	0.0	0.1	0:00.04	bash
868	root	20	0	3136	908	768	S	0.0	0.0	0:00.00	SessionLeader
869	root	20	0	3136	1172	1024	S	0.0	0.0	0:00.03	Relay(876)
876	monis	20	0	6260	5376	3712	S	0.0	0.1	0:00.15	bash
1025	polkitd	20	0	306436	8064	7424	S	0.0	0.2	0:00.19	polkitd
1605	root	20	0	1866024	14336	11904	S	0.0	0.4	0:00.14	wsl-pro-service

Press q

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
153 root      20    0   18420   8960   7936 S    0.0   0.2   0:00.17 systemd-logind
166 syslog    20    0  220532   5248   4224 S    0.0   0.1   0:00.24 rsyslogd
222 _chrony    20    0  24544   11184   6656 S    0.0   0.3   0:00.25 chronyd
229 root      20    0    5528   2688   2560 S    0.0   0.1   0:00.02 agetty
231 root      20    0 123440   31616  17280 S    0.0   0.8   0:00.50 unattended-upgr
233 _chrony    20    0   12076   2232   1664 S    0.0   0.1   0:00.15 chronyd
256 root      20    0    5484   2560   2432 S    0.0   0.1   0:00.01 agetty
315 root      20    0    3136    900    768 S    0.0   0.0   0:00.00 SessionLeader
317 root      20    0    3136   1164   1024 S    0.0   0.0   0:00.25 Relay(321)
321 monis     20    0    6260   5248   3712 S    0.0   0.1   0:00.23 bash
323 root      20    0    8632   5376   4608 S    0.0   0.1   0:00.02 login
373 monis     20    0   22492  13056  10624 S    0.0   0.3   0:00.20 systemd
375 monis     20    0   22892   4044   2048 S    0.0   0.1   0:00.00 (sd-pam)
403 monis     20    0    6128   5120   3712 S    0.0   0.1   0:00.04 bash
868 root      20    0    3136    908    768 S    0.0   0.0   0:00.00 SessionLeader
869 root      20    0    3136   1172   1024 S    0.0   0.0   0:00.03 Relay(876)
876 monis     20    0    6260   5376   3712 S    0.0   0.1   0:00.15 bash
1025 polkitd   20    0  306436   8064   7424 S    0.0   0.2   0:00.19 polkitd
1605 root      20    0 1866024  14336  11904 S    0.0   0.4   0:00.14 wsl-pro-service

monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

After 30seconds in Terminal-1

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical3
Parent process started.
Parent PID: 1601

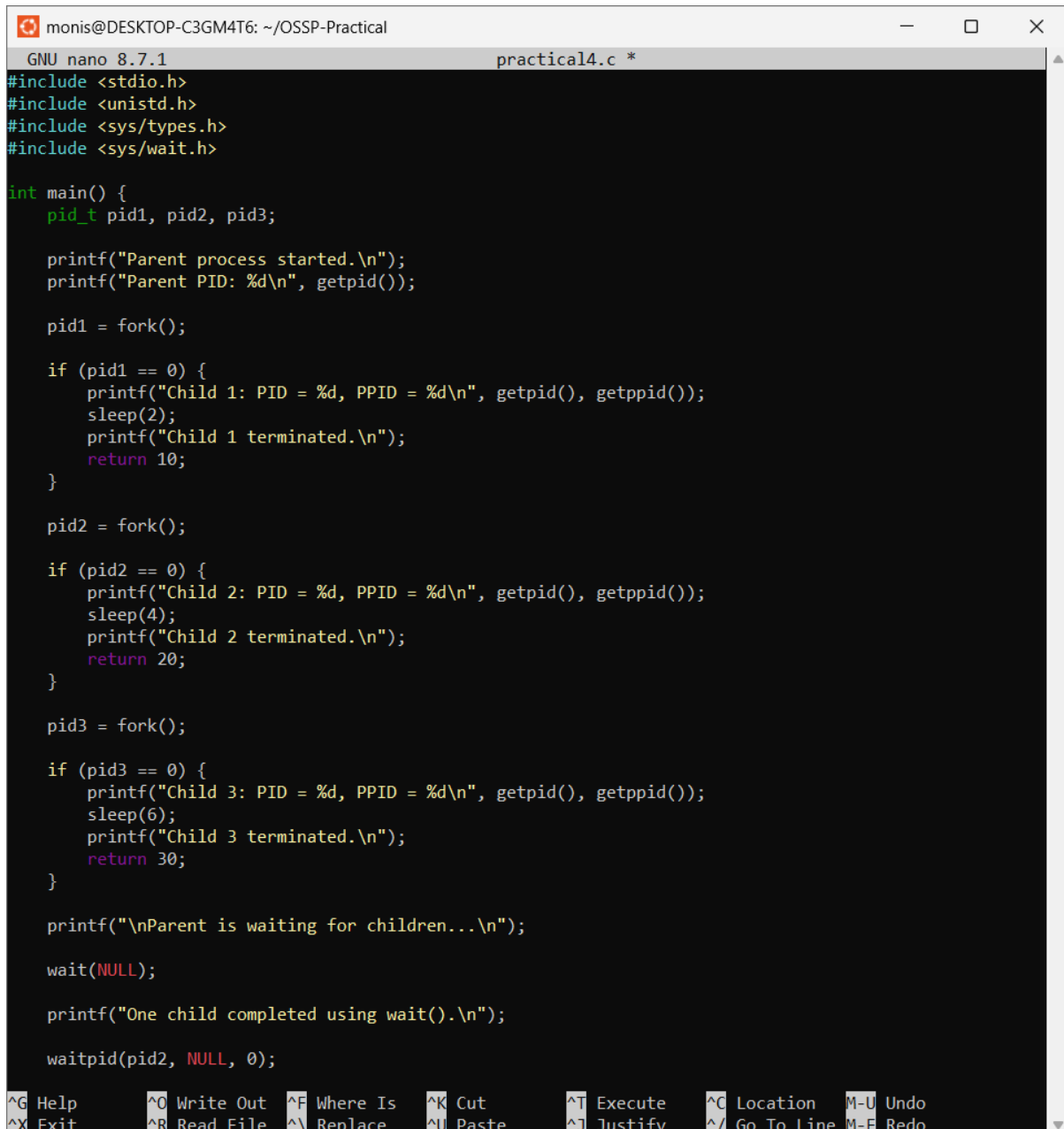
--- PARENT PROCESS ---
Parent PID: 1601
Child PID: 1602
Parent is waiting for child...

--- CHILD PROCESS ---
Child PID: 1602
Child PPID: 1601
Child is running...
Child process terminating...
Child has terminated.
Parent process terminating...
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

[`zombie.c` → demonstrates zombie creation
`zombie1.c` → demonstrates zombie removal using `wait()`]



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical4.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2, pid3;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid1 = fork();

    if (pid1 == 0) {
        printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(2);
        printf("Child 1 terminated.\n");
        return 10;
    }

    pid2 = fork();

    if (pid2 == 0) {
        printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(4);
        printf("Child 2 terminated.\n");
        return 20;
    }

    pid3 = fork();

    if (pid3 == 0) {
        printf("Child 3: PID = %d, PPID = %d\n", getpid(), getppid());
        sleep(6);
        printf("Child 3 terminated.\n");
        return 30;
    }

    printf("\nParent is waiting for children...\n");

    wait(NULL);

    printf("One child completed using wait().\n");

    waitpid(pid2, NULL, 0);

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical4.c *

if (pid1 == 0) {
    printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());
    sleep(2);
    printf("Child 1 terminated.\n");
    return 10;
}

pid2 = fork();

if (pid2 == 0) {
    printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());
    sleep(4);
    printf("Child 2 terminated.\n");
    return 20;
}

pid3 = fork();

if (pid3 == 0) {
    printf("Child 3: PID = %d, PPID = %d\n", getpid(), getppid());
    sleep(6);
    printf("Child 3 terminated.\n");
    return 30;
}

printf("\nParent is waiting for children...\n");

wait(NULL);

printf("One child completed using wait().\n");

waitpid(pid2, NULL, 0);

printf("Child 2 completed using waitpid().\n");

waitpid(pid3, NULL, 0);

printf("Child 3 completed using waitpid().\n");

printf("Parent process terminating.\n");

return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
return 0;
}

Write to File: practical4.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel    M-M Mac Format M-P Prepend  ^Q Discard buffer
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical4.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc practical4.c -o practical4
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical4
Parent process started.
Parent PID: 1795
Child 2: PID = 1797, PPID = 1795
Child 1: PID = 1796, PPID = 1795

Parent is waiting for children...
Child 3: PID = 1798, PPID = 1795
Child 1 terminated.
One child completed using wait().
Child 2 terminated.
Child 2 completed using waitpid().
Child 3 terminated.
Child 3 completed using waitpid().
Parent process terminating.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

- Parent process created
- 3 child processes created
- wait() used
- waitpid() used
- All children terminated
- Parent terminated last

Now do **Practical 4 Part 2 — Zombie Process.**

Step 1: Create zombie program

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano zombie.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 zombie.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child terminating...\n");
        return 0;
    }

    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping for 30 seconds...\n");

        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

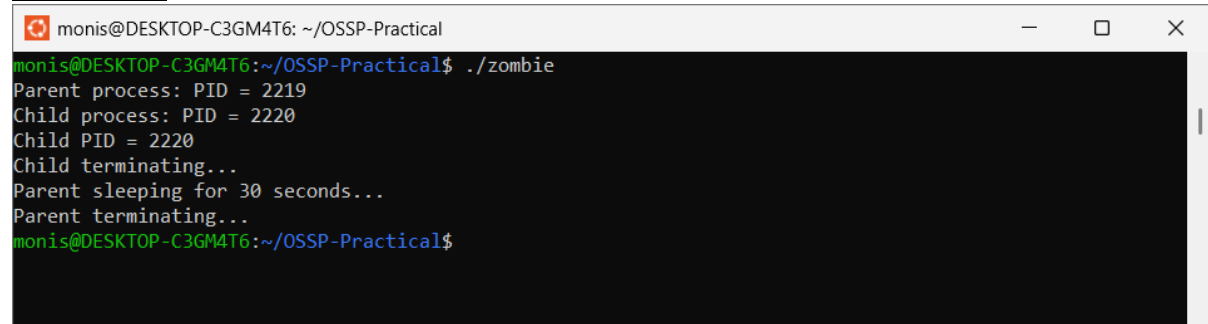
```
Write to File: zombie.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
Child 2: PID = 1797, PPID = 1795
Child 1: PID = 1796, PPID = 1795

Parent is waiting for children...
Child 3: PID = 1798, PPID = 1795
Child 1 terminated.
One child completed using wait().
Child 2 terminated.
Child 2 completed using waitpid().
Child 3 terminated.
Child 3 completed using waitpid().
Parent process terminating.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano zombie.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano zombie.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc zombie.c -o zombie
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./zombie
```

Terminal-1



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./zombie
Parent process: PID = 2219
Child process: PID = 2220
Child PID = 2220
Child terminating...
Parent sleeping for 30 seconds...
Parent terminating...
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Before 30 seconds

Immediately Terminal 2

3. Run

In **Terminal 1**:

`./zombie`

You should see:

Parent process: PID = XXXX

Child PID = YYYY

Parent sleeping for 60 seconds...

Child process: PID = YYYY

Child terminating...

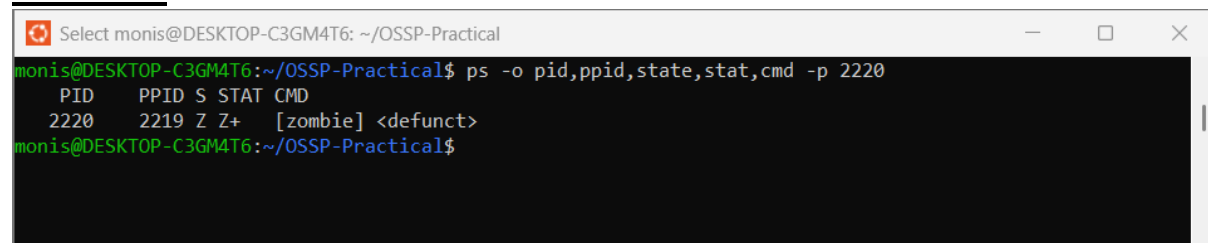
Now immediately switch to Terminal 2.

4. Check the child

If the child PID is, for example, YYYY = 2220, run:

`ps -o pid,ppid,state,stat,cmd -p 2220`

Terminal-2



```
Select monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ps -o pid,ppid,state,stat,cmd -p 2220
  PID   PPID S  STAT CMD
  2220   2219 Z  Z+   [zombie] <defunct>
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Now Create New Zombie1.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 zombie1.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child terminating...\n");
        return 0;
    }

    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent waiting for child using wait()...\n");

        wait(NULL);

        printf("Child has been collected by parent.\n");
        printf("No zombie process remains.\n");

        printf("Parent process terminating.\n");
    }

    return 0;
}
```

```
Write to File: zombie1.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Terminal-1:

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano zombie1.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc zombie1.c -o zombie1
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./zombie1
```



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./zombie1
Parent process: PID = 2412
Child PID = 2413
Parent waiting for child using wait()...
Child process: PID = 2413
Child terminating...
Child has been collected by parent.
No zombie process remains.
Parent process terminating.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Terminal-2:

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ps -o pid,ppid,state,stat,cmd -p 2413
PID    PPID S  STAT CMD
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```